

1. 删除某一个元组的值

[例 3.78] 删除学号为 20180007 的学生记录。

```
DELETE FROM Student
WHERE Sno='20180007';
```

2. 删除多个元组的值

[例 3.79] 删除所有的学生选课记录。

```
DELETE FROM SC;
```

这条 DELETE 语句将使 SC 成为空表，它删除了 SC 的所有元组。

3. 带子查询的删除语句

子查询同样也可以嵌套在 DELETE 语句中，用以构造执行删除操作的条件。

[例 3.80] 删除计算机科学与技术专业所有学生的选课记录。

```
DELETE FROM SC
WHERE Sno IN
    (SELECT Sno
     FROM Student
     WHERE Smajor= '计算机科学与技术');
```

对某个基本表中数据的更新(增、删、改)操作有可能会破坏参照完整性，第 5 章 5.3 节参照完整性将详细讲解如何进行参照完整性检查和控制。

3.5 空值的处理

前面已经多处提到空值(NULL)的概念和处理，这里再系统讲解一下什么是空值，空值如何参与运算等问题。

所谓空值就是“不知道”“不存在”或“无意义”的值。SQL 语言中允许某些元组的某些属性在一定情况下取空值。一般有以下几种情况：

① 该属性应该有一个值，但目前还不知道它的具体值。例如，某学生在填学生登记表时漏了出生日期项，不知道该学生此信息，因此出生日期属性值取空值。

② 该属性不应该有值。例如，缺考学生的成绩为空，因为他没有参加考试。

③ 由于某种原因不便于填写。例如，一个人的电话号码不想让大家知道，则取空值。

因此，空值是一个很特殊的值，含有不确定性，对关系运算带来的特殊问题需要做特殊处理。

1. 空值的产生

[例 3.81] 向 SC 表中插入一个元组，学号为“20180006”，课程号为“81004”，选课学期为

2021 年第 1 学期,选课班没有确定,没有课程成绩。

```
INSERT INTO SC(Sno,Cno,Grade,Semester,Teachingclass)
VALUES('20180006','81004',NULL,'20211',NULL);
/* 在插入时该学生还没有选定教学班,没有考试成绩,都要取空值 */
```

或

```
INSERT INTO SC(Sno,Cno,Semester)
VALUES('20180006','81004','20211');
/* 在插入语句的 INTO 子句中没有指定的属性,系统自动置空值 */
```

[例 3.82] 将 Student 表中学号为 20180006 的学生的主修专业改为空值。

```
UPDATE Student
SET Smajor = NULL
WHERE Sno='20180006';
```

另外,外连接也会产生空值,参见 3.3.2 小节例 3.55。空值的关系运算也会产生空值。

2. 空值的判断

判断一个属性的值是否为空值,用 IS NULL 或 IS NOT NULL 来表示。

[例 3.83] 从 Student 表中找出漏填了数据的学生信息。

```
SELECT *
FROM Student
WHERE Sname IS NULL OR Ssex IS NULL OR Sbirthdate IS NULL OR Smajor IS NULL;
```

WHERE 子句中没有属性 Sno,是因为 Sno 是主码,不允许取空值,不能漏填。

3. 空值约束

在创建基本表时,如果属性定义(或者域定义)为 NOT NULL 约束,则该属性不能取空值。

主码的属性不能取空值。例如 SC 表的主码是(Sno,Cno),Sno 和 Cno 都不能取空值。

Student 表的主码是 Sno,不能取空值。

在创建基本表时,如果属性定义(或者域定义)为 UNIQUE 约束,那么该属性能不能取空值呢?对于具有 UNIQUE 约束的属性,是可以取空值的。进一步,是否可以有多个记录在此属性上取空值呢?很遗憾,SQL 标准中对此没有明确规定。各个数据库管理系统在实现时就不尽相同,Oracle、KingbaseES 和 PostgreSQL 允许有多个记录在具有 UNIQUE 约束的列上取空值,而 SQL Server 则不允许。读者需要了解所使用的数据库管理系统的具体规定,实验方法可参考二维码相关内容。

4. 空值的算术运算、比较运算和逻辑运算

空值与另一个值(包括另一个空值)的算术运算的结果为空值。



扩展阅读:加了
UNIQUE 约束条
件的属性能否取
空值

空值与另一个值(包括另一个空值)的比较运算的结果为 UNKNOWN。

有了 UNKNOWN 后,传统的逻辑运算中二值(TRUE,FALSE)逻辑就扩展成了三值逻辑。逻辑运算 AND、OR、NOT 的真值表如表 3.7 所示,其中 T 表示 TRUE, F 表示 FALSE, U 表示 UNKNOWN。

表 3.7 逻辑运算 AND、OR、NOT 真值表

x	y	$x \text{ AND } y$	$x \text{ OR } y$	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

在查询语句中,只有使 WHERE 子句和 HAVING 短语中的选择条件为 TRUE 的元组才被选出作为输出结果。

[例 3.84] 找出选修 81001 号课程且成绩不及格的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='81001';
```

选出的学生是那些参加了考试而成绩不及格(Grade 属性为非空值)的学生,不包括缺考(Grade 属性为空值)的学生。因为前者使条件 Grade<60 的值为 TRUE,后者使条件的值为 UNKNOWN。

[例 3.85] 选出选修 81001 号课程且成绩不及格的学生以及缺考的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='81001'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='81001';
```

或

```
SELECT Sno
FROM SC
WHERE Cno='81001' AND (Grade < 60 OR Grade IS NULL);
```

3.6 视图

视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制。

视图是从一个或几个基本表(或视图)导出的表。它与基本表不同,是一个虚表。数据库中只存放视图的定义,而不存放视图对应的数据,这些数据仍存放在原来的基本表中。所以一旦基本表中的数据发生变化,从视图中查询出的数据也就随之改变了。从这个意义上讲,视图就像一个窗口,透过它可以看到数据库中自己感兴趣的数据及其变化。

视图一经定义,就可以和基本表一样被查询、被删除。也可以在一个视图之上再定义新的视图,但对视图的更新(增、删、改)操作则有一定的限制。

本节讨论视图的定义、操作及作用。

3.6.1 定义视图

1. 建立视图

SQL 语言用 CREATE VIEW 命令建立视图,其一般语法如下:

```
CREATE VIEW <视图名>[( <列名>[, <列名>]...)]
AS <子查询>
[WITH CHECK OPTION];
```

其中,子查询可以是任意的 SELECT 语句,是否包含 ORDER BY 子句和 DISTINCT 短语取决于具体系统的实现。

WITH CHECK OPTION 表示对视图进行 UPDATE、INSERT 和 DELETE 操作时,要保证更新、插入或删除的行满足视图定义中的谓词条件(即子查询中的条件表达式)。

组成视图的属性列名或者全部省略,或者全部指定,没有第三种选择。如果省略了视图的各个属性列名,则隐含该视图由子查询中 SELECT 子句目标列中的诸字段组成。但在下列三种情况下必须明确指定组成视图的所有列名:

- ① 某个目标列不是单纯的属性名,而是聚集函数或列表表达式。
- ② 多表连接时选出了几个同名列作为视图的字段。
- ③ 需要在视图中为某个列启用新的更合适的名字。

[例 3.86] 建立信息管理与信息系统专业学生的视图。